KLAi

Optimisation in practice: a penalty against overfitting, and numbers that must not overflow

How do we limit memorising in a model with billions of numbers, and why does a very long answer not overflow the computer?

Mathematics for AI and Machine Learning | Session 12 of 18 | Calculus and optimisation
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Fit a curve with a ridge (L2) penalty, and read what the penalty does to the weights and to the error on new points

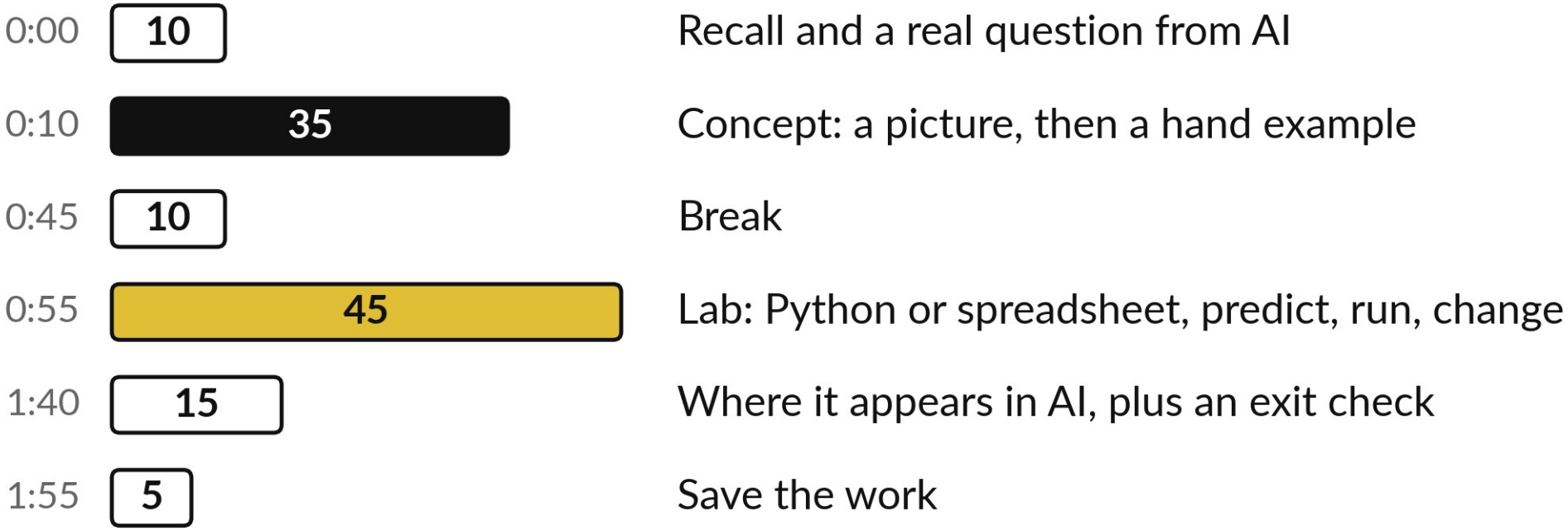


Explain a constraint as the best point on a boundary, and weight decay as the same idea written as a penalty



Compute a softmax and a log-sum-exp without overflow, by subtracting the maximum, and say why vanishing and exploding gradients happen

Your two hours: a penalty first, then numbers that break



Pictures and hand calculations first. Then the lab: numpy, or cells in the spreadsheet.

Three questions from last time

1. In our tiny network, what is delta(2) and what value did it have?

2. Why does the hidden delta contain a × (1 - a)?

3. Which values from the forward pass does the backward pass reuse?



Answer aloud first. Then we check together.

Answers to the three questions from Session 11

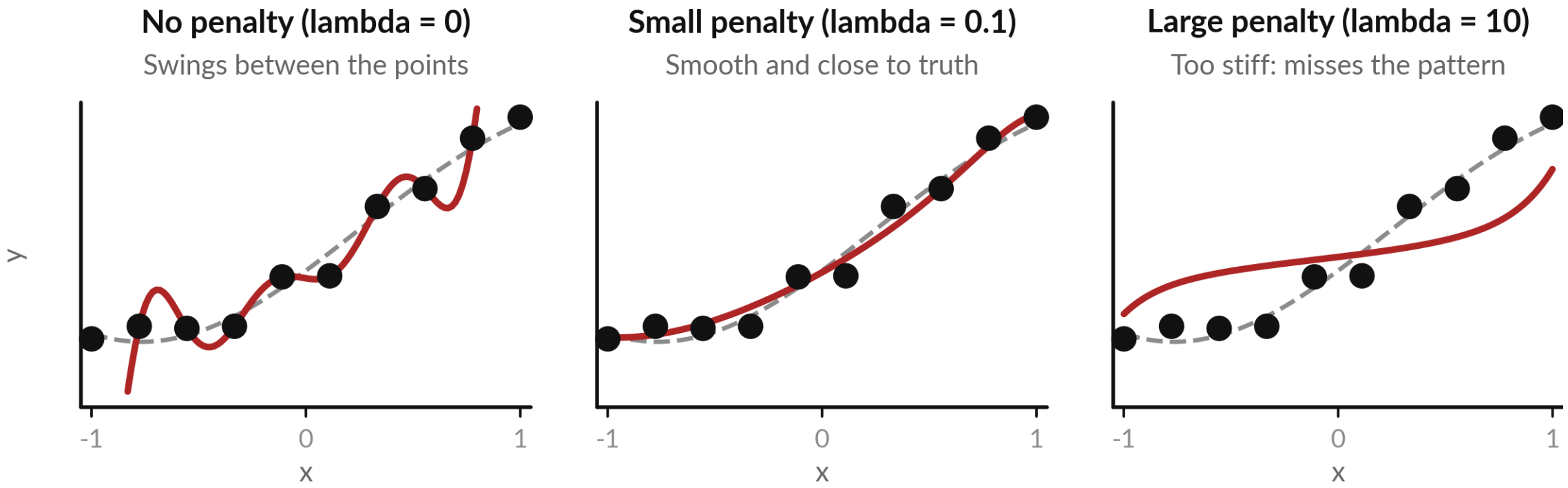
Question	Short answer
In our tiny network, what is delta(2) and what value did it have?	delta(2) is the slope of the loss with respect to the prediction, y-hat - y. It was 0.1733 - 1 = -0.8267.
Why does the hidden delta contain a × (1 - a)?	a × (1 - a) is the slope of the sigmoid. The chain rule multiplies it into the slope that arrives from the output.
Which values from the forward pass does the backward pass reuse?	The hidden outputs a(1) and the inputs x, which multiply the deltas to give the gradients. They are stored during the forward pass.

Check your answers. If one surprised you, look at Session 11 before the lab.

Words we use today

	Word	What it means
	**Regularisation**	Any change to training that discourages a model from memorising its training data.
	**L2 penalty (ridge)**	An extra term, lambda times the sum of the squared weights, added to the loss.
	**Weight decay**	Shrinking every weight a little at each update. For plain gradient descent it is the L2 penalty.
	**Overflow and underflow**	A result too large or too small for the number format: it becomes inf or 0.
	**Log-sum-exp**	The number m + ln(sum of e^(z - m)) with m the maximum. It computes ln(sum of e^z) safely.

Overfitting in one picture: a curve with too much freedom chases the noise



Dots: 10 training points. Dashed: true curve. Red: degree 9 fit, with its penalty above. Illustrative data.

The L2 (ridge) penalty adds the size of the weights to the loss

Ridge loss (sum of squared errors plus penalty)

$$L(w) = \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$$

Prediction

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \ \ldots \ + w_9 x^9$$

Symbol	What it means	In our data
lambda	The penalty strength, chosen by us. Larger means smaller weights.	0, 0.001, 0.01, 0.1, 1, 10
sum of w^2	The squared L2 norm of the weights (Session 2), without w_0.	about 104,000 at lambda = 0, 1.55 at 0.1
n, d	Number of training points; number of penalised weights.	n = 10, d = 9
y-hat	The prediction of the polynomial for a value x.	one number per point

First term: fit the data. Second term: keep the weights small. Lambda sets the balance. There is no division by n.

By hand: one weight, three points, and what lambda does to it

lambda	Formula: w = sum(x y) / (sum(x^2) + lambda)	Weight w	Share of the plain fit
0	28.5 / 14	2.0357	100%
1	28.5 / 15	1.9000	93%
14	28.5 / 28	1.0179	50%
140	28.5 / 154	0.1851	9%

Data (illustrative): x = 1, 2, 3 and y = 2.1, 3.9, 6.2, so sum(x y) = 2.1 + 7.8 + 18.6 = 28.5 and sum(x^2) = 1 + 4 + 9 = 14. Model: y-hat = w x.

Ridge has a closed-form answer: add lambda to the diagonal, then solve

Closed form

$$w = (X^T X + \lambda P)^{-1} X^T y$$

Without penalty

$$w = (X^T X)^{-1} X^T y \qquad (\lambda = 0)$$

Symbol	What it means	In our data
X	The design matrix: one row per training point, columns 1, x, x^2, ..., x^9.	10 × 10 here
X^T X	A square matrix built from X (the normal equation of Session 4).	10 × 10
lambda P	Lambda times P, added to the diagonal. P is the identity matrix with its first entry 0, so w_0 is not penalised.	lambda = 0.1
(...)^-1	The inverse (Session 4). At lambda = 0 it exists here but is badly conditioned, so the answer is unreliable.	MINVERSE in the sheet

In practice a solver is used instead of an explicit inverse. The formula shows what is computed.

Ridge on our ten points: the weights shrink, and the error on new points is lowest at lambda = 0.1

lambda	Sum of the squared weights	Training error (RMSE)	Error on 20 new points (RMSE)
0	103,994	0.0000	0.6657
0.001	3.3569	0.0890	0.1662
0.01	2.4827	0.0908	0.1636
0.1	1.5473	0.1092	0.1584
1	0.6144	0.2181	0.2039
10	0.1181	0.4549	0.4198

RMSE is the root of the mean squared error, in the units of y. The 20 new points are held out: they are never used to fit. Illustrative data.

Weight decay: every update first shrinks the weights by a small factor

L2 penalty in the update

$$w \leftarrow w - \eta\left(\frac{\partial L}{\partial w} + 2\lambda w\right)$$

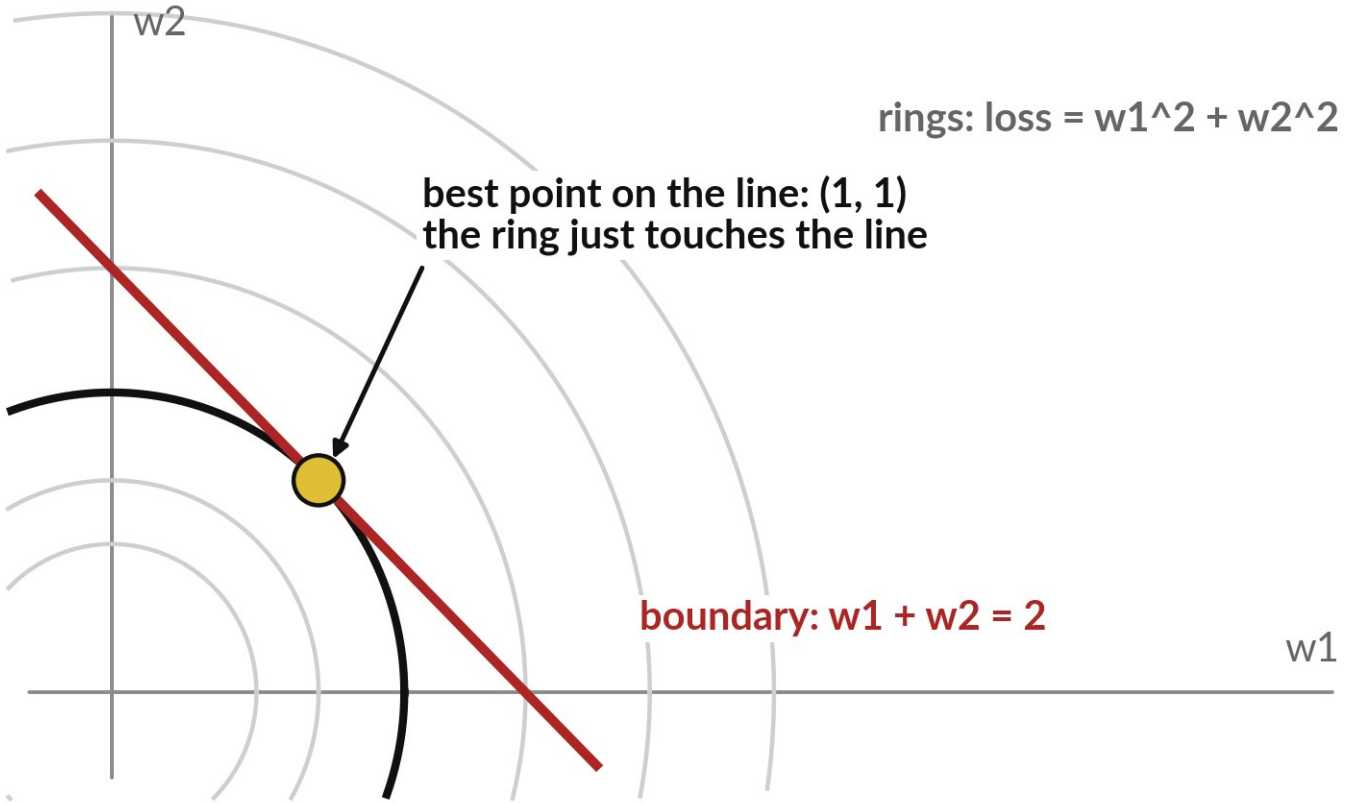
Weight decay form

$$w \leftarrow (1 - 2\eta\lambda)\, w - \eta \frac{\partial L}{\partial w}$$

Symbol	What it means	In our data
eta	The learning rate (Session 10).	eta = 0.1
dL/dw	The slope of the loss without the penalty.	illustrative: 1
lambda	The penalty strength. Many libraries define a single decay number equal to 2 times lambda.	lambda = 0.5
1 - 2 eta lambda	The shrink factor applied to w before the gradient step.	1 - 2 × 0.1 × 0.5 = 0.9

Worked: w = 2 gives 0.9 × 2 - 0.1 × 1 = 1.7. AdamW (Loshchilov and Hutter, 2017) shrinks in a separate step.

A constraint in plain words: the best point on a boundary



We want the lowest loss, but we must stay on the red line.

Grey rings: points with the same loss w1^2 + w2^2. The loss grows as the rings grow.

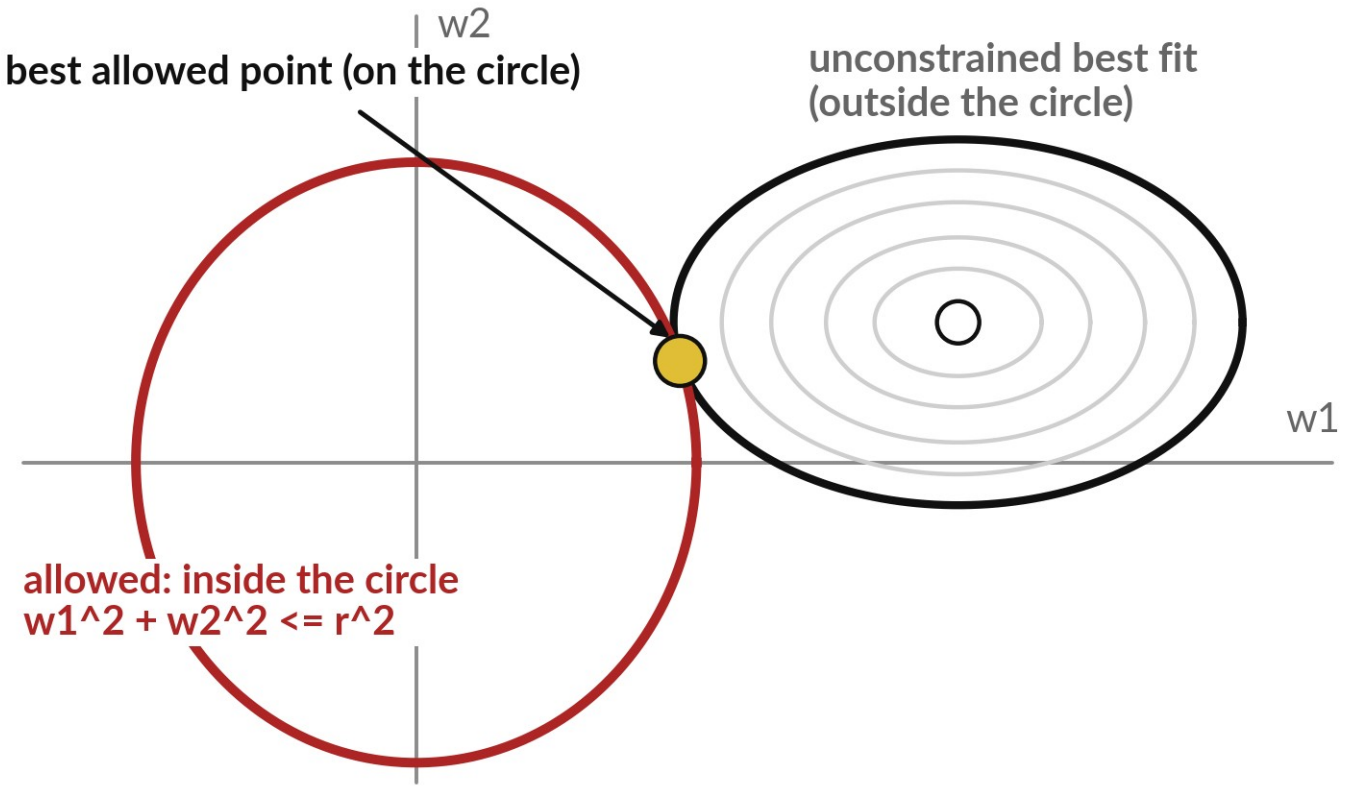
The best allowed point is where a ring just touches the line: (1, 1). The loss there is 2.

Check it by hand: walk along the line w1 + w2 = 2 and compute the loss

w1	w2 = 2 - w1	Loss w1^2 + w2^2	Calculation
0	2	4	0 + 4
0.5	1.5	2.5	0.25 + 2.25
1	1	2	1 + 1 (the lowest)
1.5	0.5	2.5	2.25 + 0.25
2	0	4	4 + 0

At the best point the gradient of the loss, (2 w1, 2 w2) = (2, 2), points in the same direction as the gradient of the constraint w1 + w2, which is (1, 1) (Session 8).

A constraint on the size of the weights: the same idea as a penalty



Ridge can be read as: find the best weights inside a circle.

The best fit with no limit lies outside the circle.

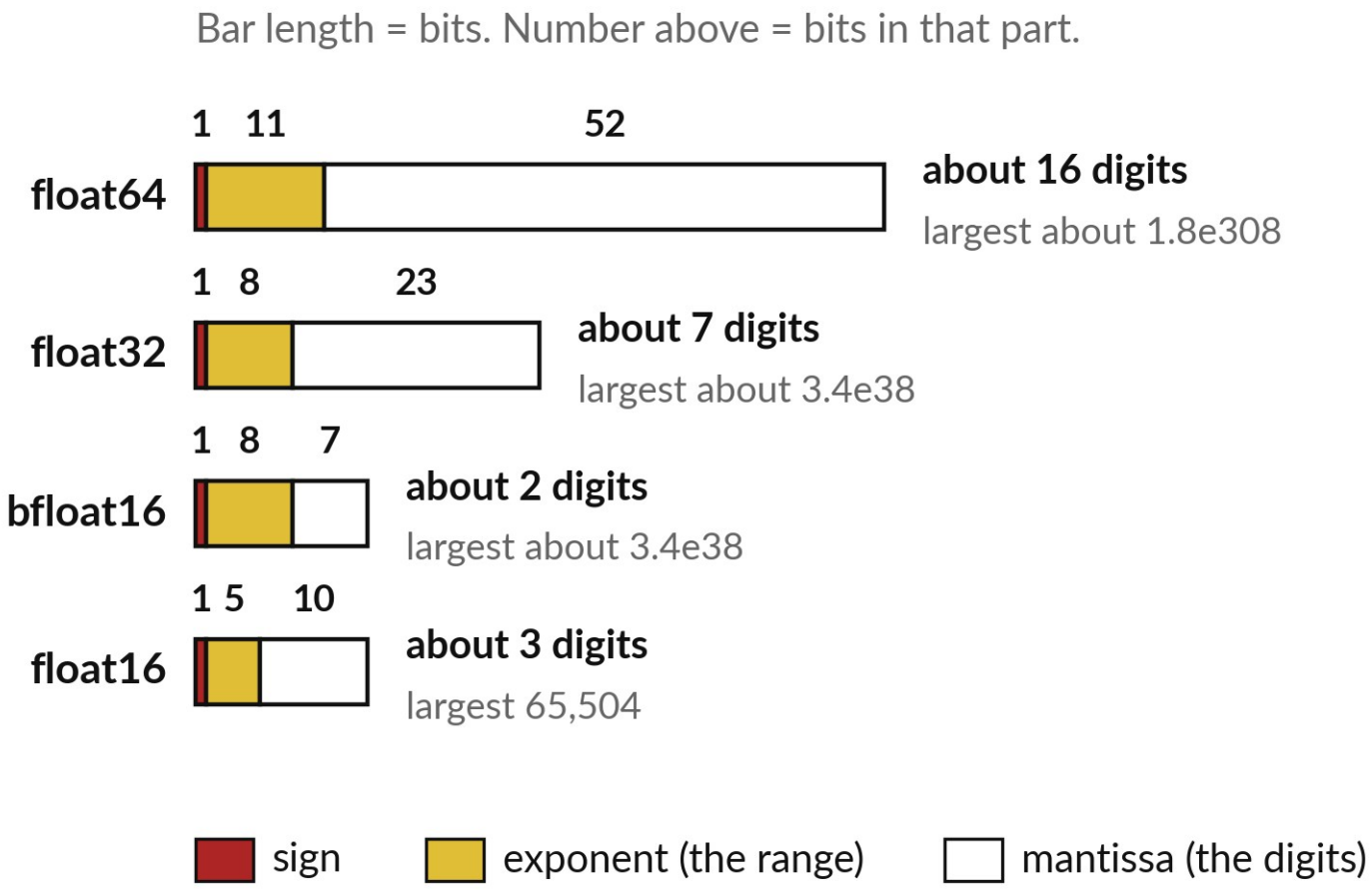
The best allowed point is on the circle, where a loss ring just touches it.

A larger lambda means a smaller circle.

A correct formula can still give nonsense on a computer

A computer stores each number in a fixed number of bits. Too large becomes inf, too small becomes 0, and inf / inf is nan (not a number).

Number formats: the bits set the range and the number of digits



Fewer bits mean a smaller range and fewer digits.

The exponent bits set how large or small a number can be. The mantissa bits set how many digits it keeps.

bfloat16 keeps the range of float32 with fewer digits. float16 has a small range: nothing above 65,504.

The exponential overflows quickly: e^z in three formats

Format	Largest value	e^z is fine at	e^z is inf at
float64 (numpy default)	about 1.8e308	z = 709: 8.2e307	z = 710
float32	about 3.4e38	z = 88: 1.65e38	z = 89
float16	65,504	z = 11: about 59,874	z = 12: 162,755 is too large

Logits (the raw scores of a model) of 100 or more are rare but possible, and a sum of exponentials is even larger than one exponential.

The stable softmax subtracts the maximum, and the answer does not change

Softmax

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Shift by a constant c

$$\frac{e^{z_i - c}}{\sum_j e^{z_j - c}} = \frac{e^{z_i}\, e^{-c}}{\sum_j e^{z_j}\, e^{-c}} = \text{softmax}(z)_i$$

Symbol	What it means	In our data
z	The logits (raw scores), one per class or candidate token.	(1000, 1001, 1002)
c	Any constant. The factor e^(-c) cancels top and bottom. Choose c = the maximum score.	c = 1002
e^(z - c)	At most 1 when c is the maximum: no overflow.	0.1353, 0.3679, 1

Softmax of (1000, 1001, 1002): the naive way fails, the stable way works

Step	Naive softmax	Stable softmax
Scores z	1000, 1001, 1002	1000, 1001, 1002
Shift by the maximum	none	z - 1002 = -2, -1, 0
Exponentials	inf, inf, inf (overflow)	0.1353, 0.3679, 1
Sum	inf	1.5032
Result	inf / inf = nan, nan, nan	0.0900, 0.2447, 0.6652

The stable result equals softmax(0, 1, 2), because only the differences between scores matter. For small scores both ways agree: softmax(2, 1, 0.1) = 0.6590, 0.2424, 0.0986.

Log-sum-exp computes the log of a sum of exponentials without overflow

Log-sum-exp

$$\mathrm{LSE}(z) = \ln \sum_j e^{z_j} = m + \ln \sum_j e^{z_j - m}, \quad m = \max_j z_j$$

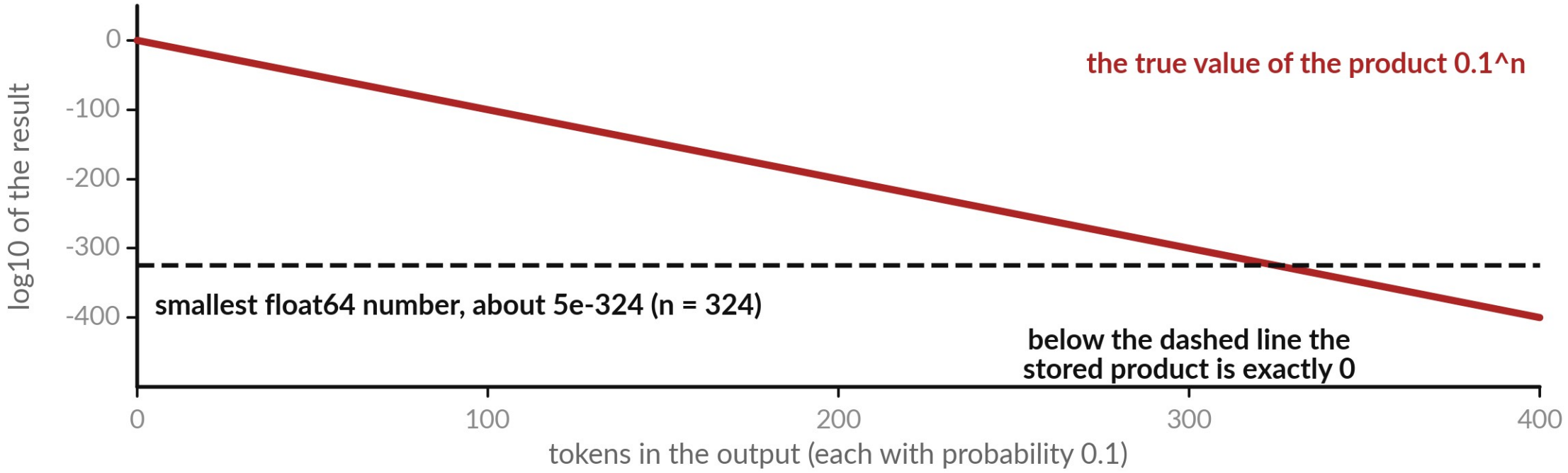
Log-softmax

$$\ln \mathrm{softmax}(z)_i = z_i - \mathrm{LSE}(z)$$

Symbol	What it means	In our data
m	The maximum of the scores. The shift keeps every exponent at most 0.	1002
LSE(z)	The log-sum-exp.	1002 + ln(1.5032) = 1002.4076
ln softmax	The log of each probability, computed directly without ever forming the probability.	-2.4076, -1.4076, -0.4076

Cross-entropy (Session 17) is the negative log-probability of the correct token, so it is computed from log-softmax this way.

A product of many probabilities underflows to zero, but the sum of their logs does not



Each token has probability 0.1, so n tokens give 0.1^n. Float64 stores 0 from n = 324.

Products and sums of logs: the same quantity, one fails and one works

Number of tokens n	Product 0.1^n	Sum of logs: n × ln(0.1)
10	1e-10	-23.026
100	1e-100	-230.26
300	1e-300	-690.78
323	9.88131e-324 (few digits left)	-743.73
324	0 (underflow)	-746.04
400	0 (underflow)	-921.03

ln(0.1) = -2.3026. The sum of logs is a perfectly ordinary number. The log-probability of a sequence is the sum of the log-probabilities of its tokens.

16-bit numbers: a saving that needs care

The gain

Half the memory of 32-bit numbers

Faster arithmetic on suitable hardware

Good results in published work

The risk

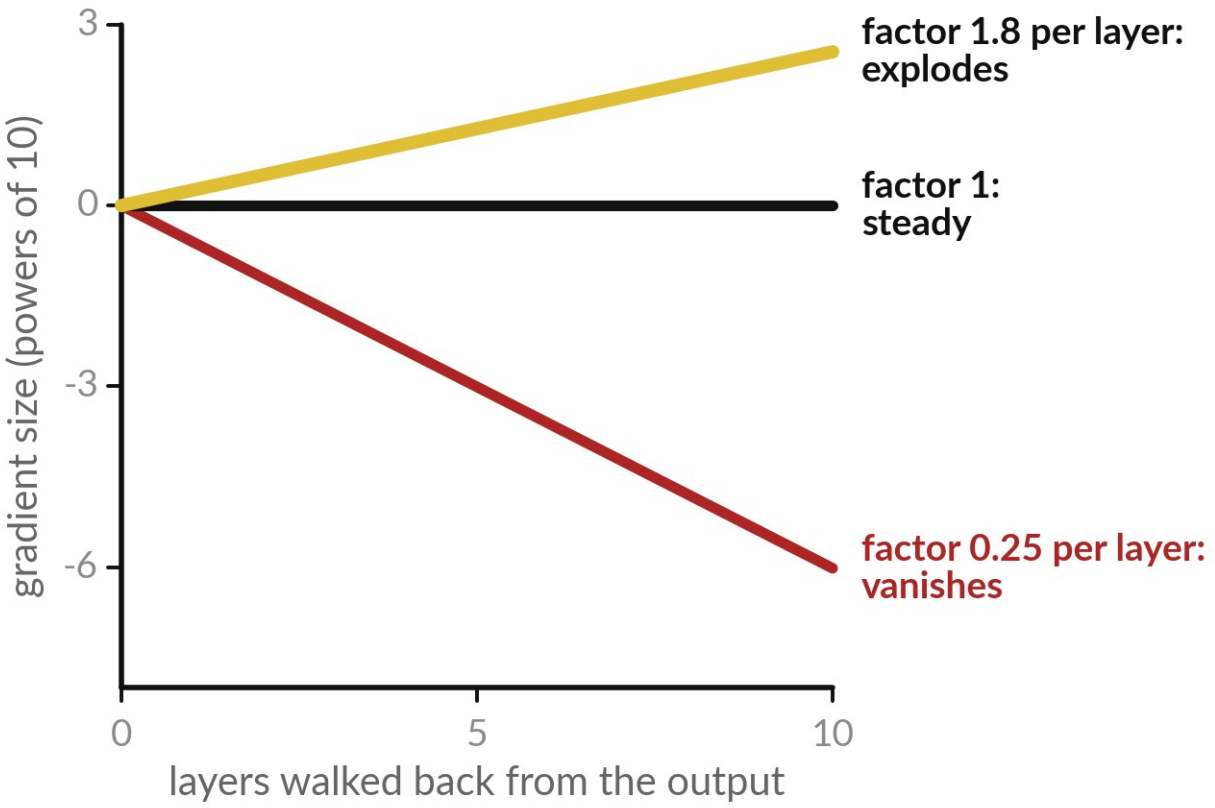
Overflow above 65,504

A small update is lost: 1 + 0.0004 is stored as 1.0

Remedy: a 32-bit weight copy and loss scaling (Micikevicius et al., 2017)

bfloat16 keeps the float32 range.

Vanishing and exploding gradients: a product of many factors goes to zero or to infinity



The backward pass multiplies one factor per layer.

Factors below 1 shrink the gradient layer by layer: it vanishes and the early layers stop learning.

Factors above 1 grow it: it explodes and the loss becomes nan.

The numbers behind the picture: a gradient after k layers, with a factor of 0.25 or 1.8

Layers k	Factor 0.25: 0.25^k	Factor 1.8: 1.8^k
1	0.25	1.8
3	0.0156	5.83
5	0.00098	18.9
10	0.00000095	357

0.25 is the largest slope a × (1 - a) of the sigmoid (Session 11). Each layer's factor also contains its weight, so 0.25^10, about one millionth, holds if each weight were 1.

Clipping a gradient limits its length when it explodes

Clip by norm

$$g \leftarrow g \cdot \frac{c}{\|g\|_2} \quad \text{if } \|g\|_2 > c$$

Symbol	What it means	In our data
g	The gradient vector of all the parameters.	g = (30, 40)
norm of g	The L2 norm (Session 2): the length of the vector.	sqrt(900 + 1600) = 50
c	The largest length we allow, chosen by us.	c = 5
New g	Multiply g by c / norm = 5 / 50 = 0.1. The direction stays the same.	(3, 4), length 5

Clipping changes the length of the step but not its direction. Gradient clipping is common in published training recipes.

Four mistakes in practice, what you see, and the fix

	Mistake	What you see	Fix
	**Exponentiate large scores directly**	inf or nan in the softmax	Subtract the maximum first, or use log-softmax
	**Pick lambda by the training error**	The best lambda is always 0	Pick lambda by the error on held-out data
	**Penalise the intercept too**	The whole curve is pulled towards 0	Leave w_0 out of the penalty
	**Multiply many probabilities**	The result is 0 for long sequences	Add log-probabilities instead

Read the stable softmax before you write it

```
z = np.array([1000., 1001., 1002.])
m = z.max()
e = np.exp(z - m)
probs = e / e.sum()
lse = m + np.log(e.sum())
logprobs = z - lse
```

In plain words

m is the largest score. z - m is at most 0.

Every entry of e is at most 1, so it cannot overflow.

probs adds up to 1 and equals softmax(0, 1, 2).

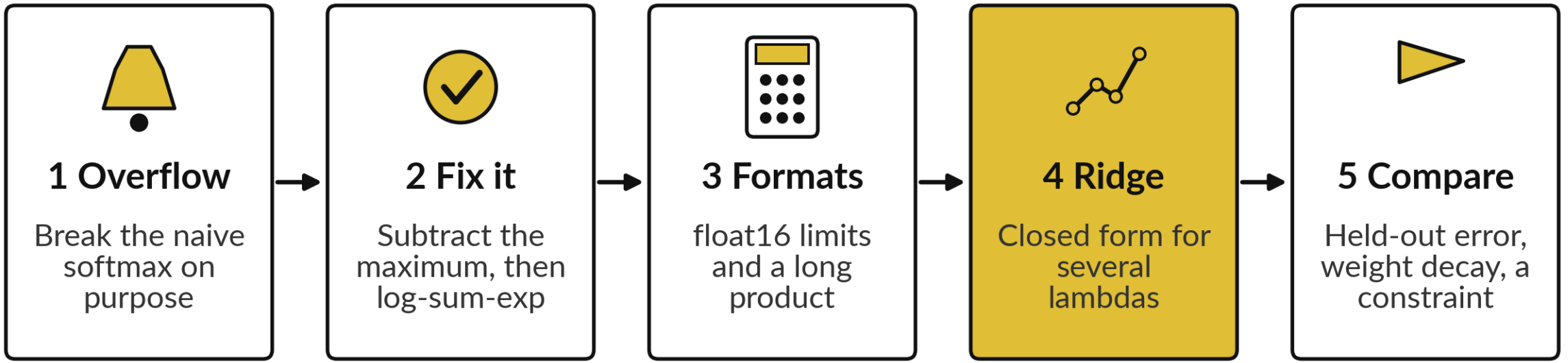
lse is the log-sum-exp. logprobs are the log of each probability, with no probability formed.



Part 2: the lab

Close the slides. Open the notebook or the workbook.

The lab in five stages, in Python and in the spreadsheet



Predict first, run, read, then change one thing. The check cells tell you when a number is right.

Check your numbers against these before you move on

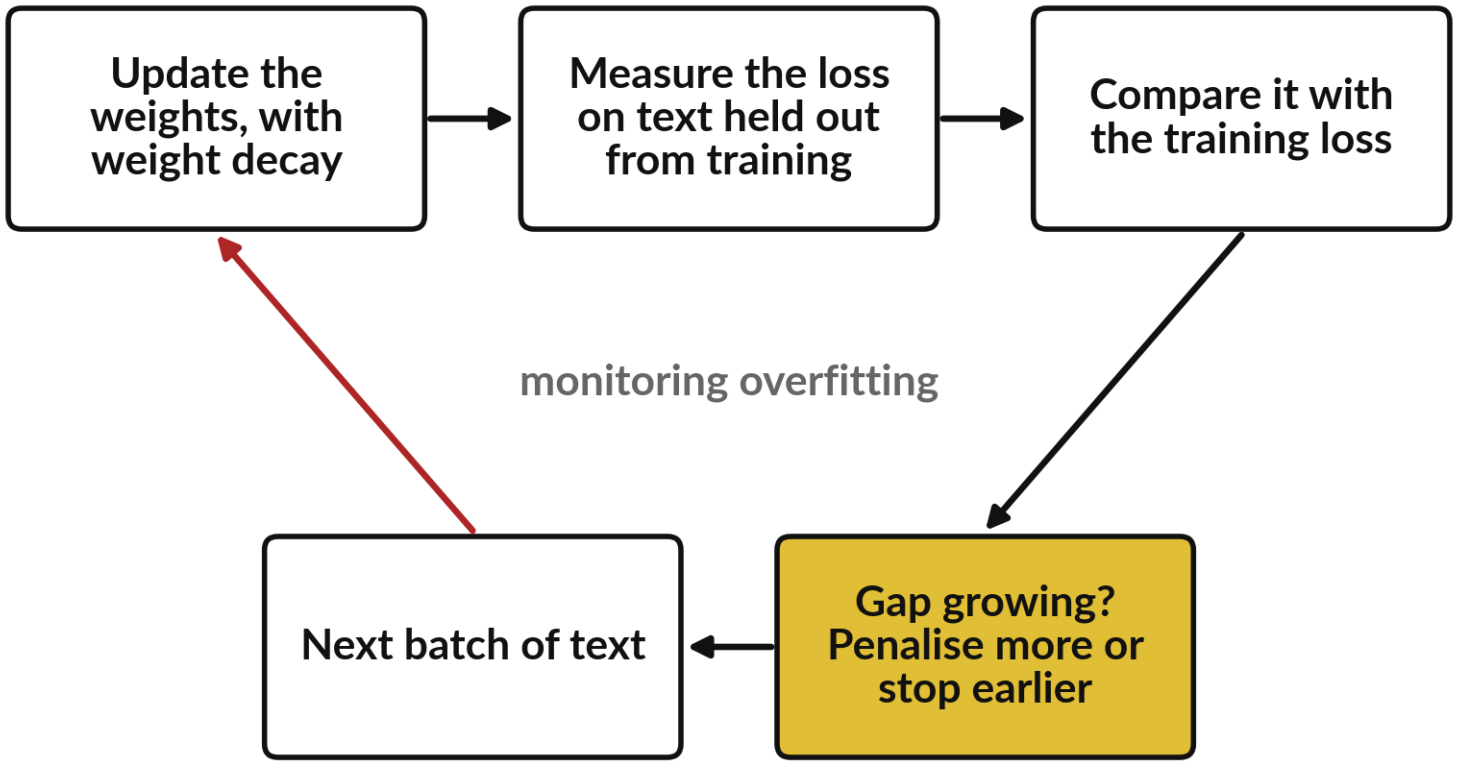
What you compute	You should see
Stable softmax of (1000, 1001, 1002)	0.0900, 0.2447, 0.6652
Log-sum-exp of (1000, 1001, 1002)	1002.4076
Largest float16 number	65504
Sum of squared weights, lambda = 0.1	1.5473
Error on the 20 new points, lambda = 0 and 0.1	0.6657 and 0.1584
Weight after one step with decay (w = 2)	1.7
Best point on the line w1 + w2 = 2	w1 = 1 and w2 = 1

If your number differs, find the step that went wrong. Do not change a number to match.

Errors you will meet in the lab, what they mean and what to do

	What you see	What it means	What to do
	**RuntimeWarning: overflow in exp, and nan**	The naive softmax overflowed. This is expected in step 1	Continue to step 2 and subtract the maximum
	**Huge or jumpy weights at lambda = 0**	lambda is 0: X^T X is badly conditioned and the answer is unreliable (numpy raises LinAlgError only if the matrix is exactly singular)	Use lambda > 0 in the closed form
	**Weights look huge**	lambda is too small	Raise lambda and compare the norm
	**#NUM! in a sheet**	EXP of a large number overflowed	Subtract the maximum first

Where this appears in AI: how overfitting is watched and controlled

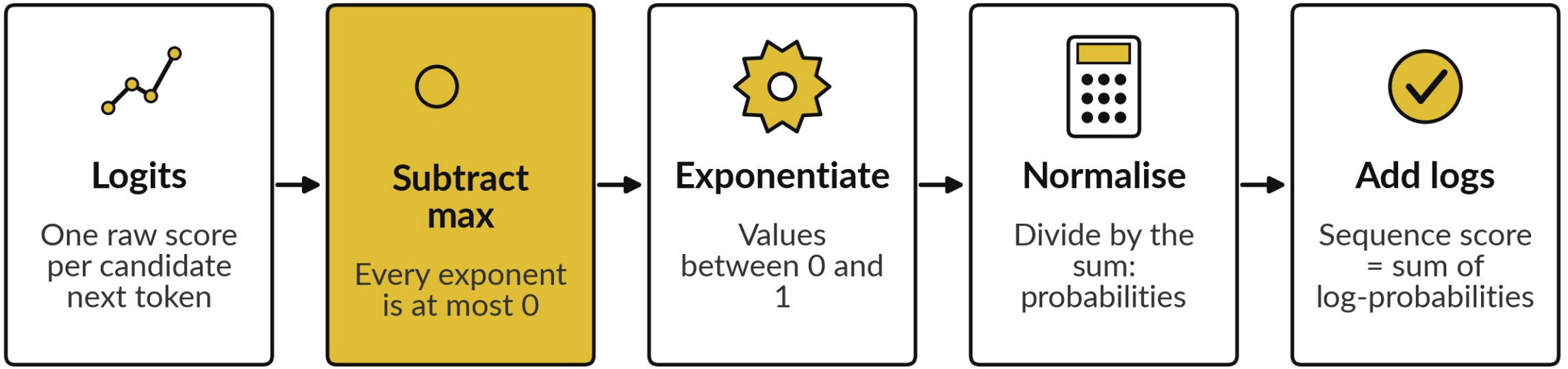


The loss on held-out text is the signal that tells us whether the model memorises.

Published work describes weight decay, dropout and early stopping for models of this kind.

The settings used by closed products are not published.

Where this appears in AI: why a very long answer does not overflow



Published practice for models of this kind: softmax computed in a stable way, and sequence scores kept as sums of logs.

Check an AI assistant with your own numbers

10 minutes Hands-on, with no private data

Ask any AI assistant to compute a softmax that overflows. Then compare with your table.

Type: "Compute softmax of the scores 1000, 1001 and 1002 without overflow. Show every step."

Expected result: 0.0900, 0.2447, 0.6652.

Did it subtract the maximum? If its answer differs, find the first number that differs from yours.

Never paste confidential data.

You keep

A note: did the assistant match your table?

Exit check: three questions with short answers

Question	Short answer
What does the L2 penalty do to the weights, and how do you choose lambda?	It shrinks the weights towards zero. Choose lambda by the error on held-out data only.
Why does subtracting the maximum not change the softmax?	The factor e^(-c) appears in the top and in every term of the bottom, so it cancels. It only keeps the exponents at most 0.
Why are log-probabilities added instead of probabilities multiplied?	A product of many probabilities underflows to 0. The sum of their logs is an ordinary number.

Save your lab and keep the two rules

What you keep from today

Keep your saved notebook or workbook, with the ridge fit for several values of lambda and the stable softmax. Write down the one-line rule: subtract the maximum before you exponentiate.

Our question today

How do we limit memorising in a model with billions of numbers, and why does a very long answer not overflow the computer?

Next session

Session 13: probability basics: random variables, expectation and variance.

Take away

Penalty
lambda × sum of w^2

Best lambda here
0.1 (test error 0.158)

Stable softmax
subtract the maximum

Next session
Probability basics